



Unidad Didáctica

Introducción a la Ingeniería del Software, Unidad I

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Origen y evolución del software.	4
La Ingeniería de Software.....	7
Desarrollo de la Ingeniería del software.....	9
Diversidad de la Ingeniería del software	11
El proceso del software	14
La ingeniería de software y la Web.....	16
Bibliografía y Webgrafía	19
Glosario	20



La Ingeniería del Software representa un pilar fundamental en el ámbito de la Ingeniería de Sistemas de Información, proyectando una influencia significativa en el futuro profesional de quienes se embarcan en esta carrera. Esta disciplina aborda la creación, desarrollo y mantenimiento de software de manera sistemática y eficiente, respondiendo a las necesidades cambiantes de la sociedad y la industria.

En esta unidad introductoria, exploraremos los fundamentos esenciales de la Ingeniería del Software. Desde su origen y evolución hasta su diversidad y aplicación en la era digital. Nos adentraremos en el proceso de desarrollo del software, destacando su importancia en la gestión de proyectos y la satisfacción de los usuarios finales.

Además, examinaremos cómo la Ingeniería del Software se entrelaza con la web, abriendo nuevas oportunidades y desafíos en un entorno cada vez más interconectado. A lo largo de este viaje, comprenderemos no solo la técnica y la metodología, sino también la relevancia social y económica de esta disciplina en constante evolución. ¡Bienvenidos a este fascinante mundo de la Ingeniería del Software!

Origen y evolución del software.

Antecedentes Precursores:

- Matemáticas y Lógica: Desde los primeros conceptos matemáticos hasta las bases de la lógica booleana, exploraremos cómo estos principios sentaron las bases para la programación. Desde los primeros registros de contabilidad en la antigua Mesopotamia hasta los desarrollos de Euclides y Arquímedes, las matemáticas proporcionaron un lenguaje universal para la comprensión de patrones y relaciones abstractas. El desarrollo de la lógica formal por parte de filósofos como Aristóteles y Leibniz estableció las reglas para el razonamiento y la inferencia, sentando así las bases para la lógica booleana de George Boole en el siglo XIX, que se convertiría en fundamental para la computación moderna.
- Máquinas Analíticas: El surgimiento de las primeras máquinas de cómputo y la contribución de pioneros como Charles Babbage y Ada Lovelace. A principios del siglo XIX, Charles Babbage concibió la idea de una "Máquina Analítica", un dispositivo mecánico capaz de realizar cálculos basados en instrucciones programadas. Aunque nunca completó su construcción, sus diseños sentaron las bases para la computación moderna. Ada Lovelace, colaboradora de Babbage, fue la primera persona en reconocer el potencial de las máquinas analíticas para algo más que simples cálculos numéricos, escribiendo algoritmos que podrían ser ejecutados por estas máquinas, lo que la convierte en la primera programadora de la historia.

La Era de los Primeros Lenguajes de Programación:

- Lenguajes de Máquina y Lenguajes Ensambladores: La programación a nivel de hardware y los desafíos asociados. Los primeros programas fueron escritos directamente en código máquina, que consistía en instrucciones binarias entendidas por las computadoras. Esta forma de programación era tediosa y propensa a errores. Los lenguajes ensambladores surgieron como una mejora, proporcionando mnemónicos que representaban instrucciones de máquina, lo que facilitaba la programación, pero aún estaba estrechamente ligada al hardware subyacente.
- Fortran y Cobol: Los primeros lenguajes de programación de alto nivel y su impacto en la computación científica y comercial. Fortran, desarrollado en la década de 1950, fue el primer lenguaje de programación de alto nivel ampliamente utilizado. Fue diseñado para la programación científica y numérica, permitiendo a los programadores expresar algoritmos de manera más clara y concisa. Cobol, también desarrollado en la década de 1950, se centró en aplicaciones comerciales, facilitando la manipulación de datos empresariales. Estos lenguajes allanaron el camino para una programación más abstracta y orientada a problemas.

El Surgimiento del Software Moderno:

- Revolución del Software: El papel crucial de los avances teóricos y prácticos, como la estructuración de programas y la modularidad. A medida que la complejidad de los programas aumentaba, se hizo evidente la necesidad de métodos para organizar y estructurar el código. Los conceptos de modularidad y abstracción se convirtieron en piedras angulares de la programación moderna, permitiendo la creación de sistemas más grandes y mantenibles. La introducción de lenguajes de programación orientados a objetos, como C++ y Java, llevó esta idea aún más lejos al permitir la encapsulación de datos y funcionalidades en objetos reutilizables.

- Los Sistemas Operativos: Desde los sistemas primitivos hasta los sistemas modernos que gestionan complejas operaciones informáticas. Los sistemas operativos evolucionaron desde simples controladores de hardware hasta complejas plataformas que gestionan recursos y ofrecen servicios a los programas. Los primeros sistemas operativos eran simples y monolíticos, pero con el tiempo se desarrollaron en sistemas más modulares y distribuidos. La introducción de sistemas como Unix y Windows marcó un hito en la historia de los sistemas operativos, estableciendo estándares para la multitarea, la interfaz de usuario y la compatibilidad de hardware.

La Era de la Computación Personal y más Allá:

- Computación Personal: El impacto de la computación personal en la democratización del software y su proliferación. La introducción de computadoras personales como la Altair 8800 y la IBM PC llevó la informática fuera de los laboratorios y las grandes corporaciones hacia los hogares y las pequeñas empresas. Esto dio lugar a una explosión de desarrollo de software, con programas que van desde aplicaciones de productividad hasta juegos y software de entretenimiento. La computación personal democratizó el acceso a la tecnología, permitiendo a individuos y pequeñas organizaciones aprovechar el poder de la informática de manera sin precedentes.
- Internet y la Nube: La evolución del software en la era digital, incluyendo la aparición de aplicaciones web y servicios en la nube. La popularización de Internet en la década de 1990 transformó la forma en que se distribuía y utilizaba el software. Las aplicaciones web, que se ejecutan en navegadores web y acceden a datos almacenados en servidores remotos, se convirtieron en una alternativa viable al software tradicional instalado localmente. La computación en la nube llevó este concepto un paso más allá, permitiendo a las organizaciones acceder a recursos informáticos escalables según sea

necesario, sin la necesidad de mantener infraestructuras costosas. Esto ha impulsado la proliferación de servicios en línea, desde redes sociales hasta almacenamiento de datos, cambiando fundamentalmente la forma en que interactuamos con la tecnología.

La Ingeniería de Software.

Definición y Principios de la Ingeniería de Software:

- Concepto y Evolución: Desde los primeros enfoques ad hoc hasta la formalización de la disciplina como Ingeniería. La Ingeniería de Software se refiere al enfoque sistemático, disciplinado y cuantificable para el desarrollo, operación y mantenimiento de software de alta calidad. Inicialmente, el desarrollo de software se realizaba de manera ad hoc, sin procesos definidos ni estándares. Con el tiempo, se reconoció la necesidad de aplicar principios de ingeniería a la construcción de software, lo que llevó a la formalización de la disciplina en la década de 1960.
- Principios Fundamentales: Abordaremos los principios básicos que guían la Ingeniería de Software, como la modularidad, la reutilización y la abstracción. La modularidad implica dividir el sistema en componentes independientes que puedan ser desarrollados, probados y mantenidos de manera separada, lo que facilita la comprensión y la gestión del software. La reutilización promueve el uso de componentes existentes en lugar de desarrollarlos desde cero, lo que ahorra tiempo y recursos. La abstracción implica ocultar los detalles de implementación y centrarse en la funcionalidad, lo que facilita la comprensión y la adaptación del software a medida que cambian los requisitos.

Proceso de Ingeniería de Software:

- Ciclo de Vida del Software: Exploraremos los diferentes modelos de ciclo de vida del software, como el modelo en cascada, el modelo en espiral y el modelo de desarrollo ágil. El modelo en cascada sigue una secuencia lineal de etapas, como requisitos, diseño, implementación, pruebas y mantenimiento. El modelo en espiral incorpora iteraciones y evaluaciones continuas para mitigar riesgos. El desarrollo ágil se basa en ciclos cortos de desarrollo iterativo e incremental, priorizando la colaboración con el cliente y la capacidad de respuesta al cambio.
- Actividades del Proceso: Analizaremos las etapas clave del proceso de Ingeniería de Software, desde la especificación de requisitos hasta la implementación, pruebas y mantenimiento. La especificación de requisitos implica identificar y documentar las necesidades del usuario. El diseño transforma los requisitos en una arquitectura de software. La implementación implica la traducción del diseño en código ejecutable. Las pruebas validan el software para garantizar su calidad y cumplimiento de los requisitos. El mantenimiento implica la corrección de errores y la incorporación de cambios en el software en funcionamiento.

Metodologías y Enfoques de Desarrollo:

- Desarrollo Tradicional: Discutiremos el enfoque secuencial del desarrollo de software y sus desafíos. El desarrollo tradicional sigue un enfoque secuencial, donde cada etapa se completa antes de pasar a la siguiente, lo que puede llevar a problemas de adaptación a cambios y retrasos en la entrega.
- Desarrollo Ágil: Exploraremos los principios y prácticas del desarrollo ágil, incluyendo Scrum, Kanban y Extreme Programming (XP). El desarrollo ágil prioriza la entrega temprana y continua de software funcional, la colaboración con el cliente y la capacidad de respuesta al cambio. Scrum divide el trabajo en iteraciones llamadas "sprints", Kanban se centra en la gestión visual del

flujo de trabajo y XP promueve prácticas como la programación en parejas y las pruebas automatizadas.

Calidad y Gestión en la Ingeniería de Software:

- Aseguramiento de la Calidad: Discutiremos la importancia de la calidad del software y las técnicas para garantizarla. El aseguramiento de la calidad implica actividades como pruebas de software, revisión de código, análisis estático y dinámico, y gestión de configuración, que buscan garantizar que el software cumpla con los requisitos y estándares de calidad establecidos.
- Gestión de Proyectos de Software: Analizaremos las mejores prácticas de gestión de proyectos aplicadas en el contexto de la Ingeniería de Software. La gestión de proyectos de software incluye la planificación, seguimiento y control de actividades para garantizar que el software se entregue dentro del tiempo, el presupuesto y los requisitos especificados. Se utilizan enfoques como la metodología de desarrollo ágil, la gestión de riesgos y la gestión de cambios para gestionar proyectos de manera efectiva.

Desarrollo de la Ingeniería del software

Tendencias Actuales:

- Desarrollo Ágil: Discutiremos la adopción generalizada de enfoques ágiles en la industria del software y cómo esto ha cambiado la forma en que se desarrollan y entregan los productos. En la actualidad, el desarrollo ágil ha ganado una amplia aceptación en la industria del software debido a su enfoque iterativo e incremental que prioriza la colaboración con el cliente, la entrega temprana y continua de software funcional, y la capacidad de respuesta al cambio. Metodologías como Scrum, Kanban y Extreme Programming (XP) han revolucionado la forma en que se gestionan los

proyectos de software, promoviendo la autoorganización de equipos, la retroalimentación constante y la mejora continua del proceso.

- DevOps: Exploraremos la integración de desarrollo y operaciones en un enfoque continuo de entrega de software, y cómo esto está transformando la forma en que se desarrolla, prueba y despliega el software. DevOps es una cultura y conjunto de prácticas que buscan integrar los equipos de desarrollo y operaciones para mejorar la colaboración, la calidad y la velocidad en la entrega de software. Automatización, pruebas continuas, integración continua (CI) y entrega continua (CD) son elementos clave de DevOps que permiten a las organizaciones desarrollar, probar y desplegar software de manera rápida y confiable.

Futuro de la Ingeniería del Software:

- Inteligencia Artificial y Machine Learning: Analizaremos cómo la IA y el aprendizaje automático están impactando la Ingeniería del Software, desde la automatización de pruebas hasta la generación automática de código. La inteligencia artificial y el aprendizaje automático están siendo cada vez más utilizados en la Ingeniería del Software para mejorar la eficiencia y la calidad del desarrollo de software. Esto incluye la automatización de tareas repetitivas como pruebas de software, la detección de errores y la generación de código, así como la optimización de procesos de desarrollo y la personalización de experiencias de usuario.
- Computación Cuántica: Exploraremos el potencial impacto de la computación cuántica en el desarrollo de software, incluyendo el diseño de algoritmos y la optimización de sistemas. La computación cuántica promete revolucionar la informática al ofrecer capacidades de procesamiento incomparables. En el campo de la Ingeniería del Software, la computación cuántica podría conducir a avances significativos en el diseño de algoritmos más eficientes para resolver problemas complejos, así como en la optimización de sistemas y la

seguridad de la información. Sin embargo, aún estamos en las etapas iniciales de exploración de cómo la computación cuántica impactará la práctica de la Ingeniería del Software.

Diversidad de la Ingeniería del software

Metodologías y Enfoques de Desarrollo:

- Desarrollo Tradicional: Exploraremos enfoques secuenciales como el modelo en cascada y el modelo en V, destacando sus características y aplicaciones. El modelo en cascada sigue una secuencia lineal de etapas, donde cada fase debe completarse antes de pasar a la siguiente, comenzando con la especificación de requisitos, seguida del diseño, la implementación, las pruebas y el mantenimiento. El modelo en V también sigue un enfoque secuencial, pero enfatiza la relación entre las fases de desarrollo y las pruebas correspondientes. Estos enfoques son adecuados para proyectos con requisitos estables y bien definidos, pero pueden tener dificultades para adaptarse a cambios en los requisitos durante el desarrollo.
- Desarrollo Ágil: Analizaremos los principios y prácticas del desarrollo ágil, incluyendo Scrum, Kanban, Extreme Programming (XP) y otras metodologías ágiles emergentes. El desarrollo ágil se caracteriza por entregas iterativas e incrementales, colaboración estrecha con el cliente y capacidad de respuesta al cambio. Scrum se centra en equipos autoorganizados que trabajan en iteraciones cortas llamadas "sprints", mientras que Kanban se basa en la gestión visual del flujo de trabajo. XP promueve prácticas como la programación en parejas y las pruebas automatizadas. Estas metodologías ágiles son ideales para proyectos donde los requisitos son propensos a cambios y se prioriza la entrega rápida de valor al cliente.
- Enfoques Híbridos: Discutiremos la combinación de enfoques tradicionales y ágiles en la práctica, y cómo estas fusiones pueden adaptarse a diferentes

contextos y necesidades. Los enfoques híbridos permiten a las organizaciones aprovechar los aspectos beneficiosos tanto de los enfoques tradicionales como ágiles. Por ejemplo, un proyecto puede utilizar la metodología en cascada para la fase inicial de planificación y diseño, y luego cambiar a un enfoque ágil para la implementación y pruebas. Estos enfoques híbridos pueden proporcionar flexibilidad mientras se mantienen ciertos niveles de control y previsibilidad.

Áreas de Aplicación y Especialización:

- Desarrollo de Software Empresarial: Exploraremos la ingeniería de software enfocada en el desarrollo de aplicaciones empresariales, sistemas de gestión empresarial (ERP) y sistemas de planificación de recursos (CRM). El desarrollo de software empresarial se centra en la creación de soluciones informáticas para mejorar la eficiencia y la productividad en entornos empresariales. Esto puede incluir el desarrollo de sistemas de gestión de inventario, sistemas de recursos humanos y aplicaciones de análisis de datos para ayudar a las empresas a tomar decisiones informadas.
- Desarrollo de Software Embebido: Analizaremos la ingeniería de software aplicada al desarrollo de sistemas embebidos en dispositivos electrónicos, automóviles, dispositivos médicos y otros dispositivos especializados. El desarrollo de software embebido se enfoca en la creación de software para sistemas integrados en hardware específico, como sistemas de control de automóviles, dispositivos médicos implantables y dispositivos IoT (Internet of Things). Esto puede implicar el desarrollo de software de bajo nivel para controlar hardware, así como la implementación de algoritmos específicos para aplicaciones especializadas.
- Desarrollo de Software para la Web: Discutiremos las prácticas y tecnologías utilizadas en el desarrollo de aplicaciones web, incluyendo frameworks, tecnologías front-end y back-end, y consideraciones de seguridad. El

desarrollo de software para la web se enfoca en la creación de aplicaciones y servicios accesibles a través de navegadores web. Esto puede incluir el desarrollo de sitios web estáticos y dinámicos, aplicaciones web de una sola página (SPA) y servicios web API. Las tecnologías comunes utilizadas incluyen HTML, CSS, JavaScript, frameworks como React y Angular para el front-end, y tecnologías como Node.js, Django y Ruby on Rails para el back-end.

Roles Profesionales y Equipos de Trabajo:

- Ingenieros de Software: Describiremos las responsabilidades y habilidades necesarias para ejercer como ingeniero de software en diferentes contextos y organizaciones. Los ingenieros de software son responsables de diseñar, desarrollar, probar y mantener software de alta calidad. Esto puede implicar la escritura de código, la colaboración con otros miembros del equipo, la resolución de problemas técnicos y el aseguramiento de la calidad del software.
- Equipos Multidisciplinarios: Analizaremos cómo se estructuran y funcionan los equipos de desarrollo de software, incluyendo roles como desarrolladores, analistas de negocio, diseñadores de experiencia de usuario (UX) y testers. Los equipos de desarrollo de software suelen ser multidisciplinarios, con miembros con diferentes habilidades y roles. Esto puede incluir desarrolladores de software que escriben código, analistas de negocio que comprenden los requisitos del cliente, diseñadores de UX que diseñan la interfaz de usuario, y testers que prueban y validan el software. La colaboración efectiva entre estos roles es fundamental para el éxito del proyecto.

El proceso del software

Áreas de Especialización:

- Ingeniería de Requisitos: El proceso de identificar, analizar y documentar los requisitos del software. La ingeniería de requisitos se enfoca en comprender las necesidades de los usuarios y transformarlas en especificaciones claras y alcanzables para el desarrollo de software. Esto incluye técnicas como entrevistas con los usuarios, análisis de casos de uso y creación de documentos de requisitos.
- Ingeniería de Software Empírica: La aplicación de métodos científicos para investigar y mejorar los procesos de desarrollo de software. La ingeniería de software empírica utiliza principios de la ciencia empírica para estudiar y mejorar la práctica de la ingeniería de software. Esto puede incluir experimentos controlados, análisis estadístico de datos de desarrollo de software y la aplicación de teorías científicas al proceso de desarrollo.
- Ingeniería de Software de Sistemas Críticos: El desarrollo de software para sistemas críticos en términos de seguridad, fiabilidad y rendimiento. La ingeniería de software de sistemas críticos se centra en garantizar que el software cumpla con estándares rigurosos de seguridad y fiabilidad. Esto puede incluir sistemas utilizados en aviones, centrales nucleares, sistemas de control de tráfico aéreo y dispositivos médicos.
- Ingeniería de Software de Entretenimiento: El desarrollo de software para aplicaciones de entretenimiento, como videojuegos y simuladores. La ingeniería de software de entretenimiento se centra en crear experiencias interactivas y atractivas para los usuarios. Esto puede incluir el diseño de motores de juego, la programación de la lógica del juego y la optimización del rendimiento para diferentes plataformas.
- Ingeniería de Software de Aplicaciones Web: El desarrollo de software para aplicaciones web y plataformas en línea. La ingeniería de software de

aplicaciones web se enfoca en la creación de sitios web dinámicos, aplicaciones web y servicios en línea. Esto puede incluir el desarrollo del lado del servidor utilizando tecnologías como Node.js o Python, así como el desarrollo del lado del cliente con frameworks como React.js o Angular.

Metodologías de Desarrollo:

- Desarrollo Tradicional: El enfoque secuencial del desarrollo de software, como el modelo en cascada. En el desarrollo tradicional, las fases de desarrollo, como la especificación, el diseño, la implementación, las pruebas y el mantenimiento, se llevan a cabo en secuencia, con cada fase dependiendo del resultado de la anterior.
- Desarrollo Ágil: La filosofía ágil y sus diferentes metodologías, como Scrum, Kanban y Extreme Programming (XP). El desarrollo ágil se centra en la entrega iterativa e incremental de software, la colaboración con el cliente y la capacidad de respuesta al cambio.
- Desarrollo Basado en Componentes: El enfoque de desarrollo que se centra en la reutilización de componentes de software existentes. En el desarrollo basado en componentes, los sistemas de software se construyen ensamblando componentes reutilizables en lugar de escribir todo el código desde cero. Esto puede aumentar la productividad y la calidad del software al permitir la reutilización de componentes probados.

Aplicaciones Prácticas:

- Industria del Software: Aplicaciones de la Ingeniería del Software en la industria de la tecnología, incluyendo el desarrollo de sistemas operativos, aplicaciones empresariales y herramientas de productividad.
- Salud y Medicina: El uso de la Ingeniería del Software en el desarrollo de sistemas de información médica, registros electrónicos de salud y aplicaciones de telemedicina.

- Automatización Industrial: La aplicación de la Ingeniería del Software en sistemas de control y automatización industrial, como sistemas de control de procesos y robots industriales.

La ingeniería de software y la Web

Patrones de Diseño Web: Exploración de los patrones comunes utilizados en el diseño y desarrollo de aplicaciones web, como:

- MVC (Modelo-Vista-Controlador): Un patrón que separa la lógica de la aplicación en tres componentes principales: el Modelo, que representa los datos y la lógica de negocio; la Vista, que muestra la interfaz de usuario; y el Controlador, que maneja las interacciones del usuario y actualiza el modelo y la vista según sea necesario.
- MVVM (Modelo-Vista-Modelo de Vista): Similar al MVC, pero con una capa adicional llamada Modelo de Vista, que actúa como un intermediario entre el Modelo y la Vista. Esta capa se encarga de la lógica de presentación y la comunicación entre el Modelo y la Vista, facilitando la separación de preocupaciones y mejorando la modularidad.
- Patrón de Repositorio: Un patrón que abstrae el acceso a los datos, proporcionando una capa de abstracción entre la lógica de la aplicación y el almacenamiento de datos subyacente. Esto permite una mayor flexibilidad y facilidad de mantenimiento al cambiar entre diferentes fuentes de datos sin afectar la lógica de la aplicación.

APIs Web (Application Programming Interfaces): Estudio de las interfaces de programación de aplicaciones utilizadas para permitir la comunicación entre diferentes componentes de software en el desarrollo web, incluyendo:

- **RESTful APIs:** Un estilo de diseño de API que utiliza los métodos HTTP estándar (GET, POST, PUT, DELETE) para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) en recursos. Utiliza una arquitectura basada en recursos y proporciona una interfaz uniforme para acceder y manipular datos.
- **GraphQL:** Un lenguaje de consulta y un entorno de tiempo de ejecución que permite a los clientes especificar exactamente qué datos necesitan. Proporciona una forma eficiente de recuperar datos personalizados y reduce la sobrecarga de red al evitar la sobre-solicitud de datos.

Microservicios: Análisis de la arquitectura de microservicios en el contexto del desarrollo web, incluyendo:

- División de aplicaciones en componentes independientes: La arquitectura de microservicios divide una aplicación monolítica en servicios independientes y autónomos, cada uno con su propia base de código y responsabilidades específicas.
- Implementación y gestión: Los microservicios se implementan y gestionan de forma independiente, lo que permite escalar y actualizar componentes individuales sin afectar al resto de la aplicación. Se pueden implementar utilizando contenedores y orquestadores como Docker y Kubernetes.

Contenedores y Orquestación (Docker, Kubernetes): Investigación sobre las tecnologías de contenedores y orquestación utilizadas en el desarrollo web para la implementación y gestión eficiente de aplicaciones en entornos de producción.

Desarrollo de Aplicaciones Web Progresivas (PWA): Exploración de las técnicas utilizadas para crear aplicaciones web que ofrecen una experiencia similar a la de las aplicaciones nativas, incluyendo:

- Capacidad de funcionar sin conexión: Las PWAs utilizan tecnologías como Service Workers para permitir que las aplicaciones funcionen sin conexión, almacenando en caché recursos y datos necesarios para la funcionalidad básica.
- Acceso a dispositivos nativos: Utilizan API web como Web App Manifest y Web APIs para acceder a funcionalidades del dispositivo, como la cámara, el GPS y las notificaciones push.

Integración Continua y Despliegue Continuo (CI/CD): Estudio de las prácticas y herramientas utilizadas en el desarrollo web para automatizar el proceso de integración, prueba y despliegue de código en entornos de producción.

Arquitecturas Sin Servidor (Serverless): Análisis de la arquitectura sin servidor en el desarrollo web, incluyendo su implementación y gestión de servicios basados en eventos y la escalabilidad automática.

Interacción con APIs Externas (Consumo de APIs): Exploración de las técnicas y prácticas utilizadas para interactuar con APIs externas en el desarrollo web, incluyendo la autenticación, la autorización y la gestión de errores.

Accesibilidad Web: Investigación sobre las mejores prácticas y técnicas para garantizar que las aplicaciones web sean accesibles para todos los usuarios, incluyendo aquellos con discapacidades visuales, auditivas o motoras.

Internacionalización y Localización (i18n y l10n): Estudio de las técnicas utilizadas para internacionalizar y localizar aplicaciones web, incluyendo la traducción de contenido y la adaptación a diferentes culturas y idiomas.

Bibliografía y Webgrafía

- *Montilva, J. C., & Barrios, J. A.* (2021). **Ingeniería del Software: Un enfoque basado en procesos** (Edición en Español). Universidad de Los Andes.
- *Martel, A.* (2014). **Gestión práctica de proyectos con Scrum: Desarrollo de software ágil para el Scrum Master** (Aprender a ser mejor gestor de proyectos) (3rd ed., Edición en Español). CreateSpace Independent Publishing Platform.
- *Simões, G. S., & Vazquez, C. E.* (2018). **Ingeniería de Requisitos: Software Orientado al Negocio** (Spanish Edition). Independently published.
- *Pressman, R. S.* (2010). **Ingeniería de Software** (Spanish Edition) (7th ed.). McGraw-Hill Interamericana Editores S.A. de C.V.
- *McCarthy, R.* (2020). **Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum** (Spanish Edition). Independently published.

Glosario

- **Accesibilidad Web:** La práctica de asegurar que los sitios web y aplicaciones sean accesibles para todos los usuarios, incluidos aquellos con discapacidades visuales, auditivas o motoras.
- **APIs Web** (Application Programming Interfaces): Interfaces de programación de aplicaciones utilizadas para permitir la comunicación entre diferentes componentes de software en el desarrollo web.
- **Arquitecturas Sin Servidor** (Serverless): Un enfoque de desarrollo en el que la infraestructura subyacente es gestionada por el proveedor de servicios en la nube, permitiendo a los desarrolladores centrarse en escribir código sin preocuparse por la gestión de servidores.
- **Contenedores y Orquestación** (Docker, Kubernetes): Tecnologías utilizadas en el desarrollo web para la implementación y gestión eficiente de aplicaciones en entornos de producción mediante el uso de contenedores para encapsular y distribuir aplicaciones y orquestadores para gestionar y escalar contenedores.
- **Desarrollo Ágil:** Filosofía de desarrollo de software que se centra en la entrega iterativa e incremental, la colaboración con el cliente y la capacidad de respuesta al cambio.
- **Desarrollo Basado en Componentes:** Enfoque de desarrollo que se centra en la reutilización de componentes de software existentes para construir sistemas de software.
- **Desarrollo de Aplicaciones Web Progresivas** (PWA): Técnicas utilizadas para crear aplicaciones web que ofrecen una experiencia similar a las aplicaciones nativas, incluida la capacidad de funcionar sin conexión y acceder a dispositivos nativos.
- **Desarrollo Tradicional:** Enfoque secuencial del desarrollo de software, como el modelo en cascada, que sigue una secuencia lineal de etapas.

- **Ingeniería de Requisitos:** Proceso de identificar, analizar y documentar los requisitos del software.
- **Ingeniería de Software:** Disciplina que se ocupa de todos los aspectos del desarrollo de software, desde la concepción de la idea hasta la entrega y mantenimiento del producto final.
- **Microservicios:** Arquitectura de software que divide una aplicación monolítica en servicios independientes y autónomos, cada uno con su propia base de código y responsabilidades específicas.
- **MVC** (Modelo-Vista-Controlador): Patrón de diseño que separa la lógica de la aplicación en tres componentes principales: el Modelo, la Vista y el Controlador.
- **MVVM** (Modelo-Vista-Modelo de Vista): Patrón de diseño similar al MVC, pero con una capa adicional llamada Modelo de Vista que actúa como un intermediario entre el Modelo y la Vista.
- **Patrones de Diseño Web:** Soluciones probadas para problemas comunes en el diseño y desarrollo de aplicaciones web.
- **Repositorio:** Patrón de diseño que abstrae el acceso a los datos, proporcionando una capa de abstracción entre la lógica de la aplicación y el almacenamiento de datos subyacente.
- **RESTful APIs:** Estilo de diseño de API que utiliza los métodos HTTP estándar para realizar operaciones CRUD en recursos.
- **Tecnologías de Contenedores:** Herramientas como Docker que permiten encapsular y distribuir aplicaciones junto con sus dependencias en contenedores ligeros y portátiles.